

IntALNS: Interpretable LLM-Agent-Controlled Adaptive Large Neighborhood Search for Cooperative Marine Monitoring Task Allocation

ABSTRACT

Cooperative marine monitoring requires joint task allocation and visit sequencing under heterogeneous platform capabilities, service-start time windows, and transfer-energy limits. Changing routes alter task insertion opportunities, requiring decisions that respond to the current allocation. We propose Interpretable Agent-Controlled Adaptive Large Neighborhood Search (IntALNS), which uses allocation observations, search-execution feedback, and strategy examples to generate neighborhood controls with explicit decision rationales. The large language model agent determines how search proceeds, while ALNS reconstructs routes, enforces feasibility, and returns execution outcomes for subsequent decisions. Experiments compare IntALNS with an adaptive ALNS baseline on synthetic monitoring instances under the same search budget. Across 45 conditions, IntALNS reduces mean priority loss by 49.69% and increases complete allocations from 16 to 25. Search trajectories show sustained gains, and a recorded case connects allocation evidence, the agent’s rationale, and executed controls. These results show that IntALNS improves the allocation of important monitoring tasks while providing an interpretable link between allocation bottlenecks, agent rationales, and executed search controls.

Keywords cooperative marine monitoring, task allocation, large language model agent, adaptive large neighborhood search, interpretable search control

1. INTRODUCTION

Cooperative marine monitoring assigns spatially dispersed tasks with varied payload requirements to heterogeneous platforms, as in air-sea environmental monitoring.¹ A feasible allocation must match tasks to capabilities, respect service-start time windows, and retain sufficient transfer energy for subsequent visits and return to base. These constraints couple task-to-platform assignment with visit order, requiring allocation and route planning to be considered jointly.²

Constrained task allocation requires feasible assignments and coordinated execution.³ Auctions coordinate assignments through bids and task bundles,⁴ while evolutionary search addresses capability and timing restrictions for heterogeneous surface vehicles.⁵ Adaptive large neighborhood search (ALNS) improves routes by removing and reinserting tasks, adapting operator sampling according to past rewards.⁶ These rewards describe which changes have been productive, whereas current task insertability identifies requests that remain difficult to allocate. The two sources of information are complementary and can jointly direct route reconstruction.

Learning-based search relates the evolving solution to neighborhood control. Graph networks have selected removals while an optimization solver repairs routes,⁷ and reinforcement learning has adapted ALNS operators, removal strength, and acceptance settings.⁸ Building on this state-driven approach, a large language model (LLM) can interpret allocation observations alongside previous actions and outcomes, then propose controls that a route solver executes while enforcing feasibility.

LLMs have been used to generate candidate solutions from previous evaluations,⁹ improve executable heuristics through reflective feedback,¹⁰ and develop scheduling rules that adjust neighborhood weights during search.¹¹ In marine task allocation, the changing relationship between unassigned tasks and platform routes calls for translating allocation difficulties into a search focus and executable neighborhood operations.

We propose IntALNS, Interpretable Agent-Controlled Adaptive Large Neighborhood Search, to generate state-dependent neighborhood controls and explicit rationales from allocation observations, execution history, and strategy examples. The LLM agent interprets the evolving allocation state and directs search, while ALNS reconstructs feasible routes and returns execution feedback. Experiments evaluate allocation quality and search progress on synthetic marine monitoring instances; a recorded case examines the link between the observed state, agent rationale, executed control, and subsequent outcome.

2. PROBLEM FORMULATION

Consider a known batch of monitoring tasks planned centrally for heterogeneous platforms with deterministic inputs. Each assigned task is executed without interruption by one platform. Task $i \in \mathcal{T}$ has a location, required payload and operating capabilities $\mathcal{R}_i$, service-start time window $[e_i, l_i]$, duration s_i , and priority $p_i \geq 0$. Platform $a \in \mathcal{A}$ has capabilities $\mathcal{C}_a$, speed $v_a > 0$, transfer-energy rate ε_a per distance unit, and energy budget $E_a^{\max}$. All platforms depart from a common base, denoted 0, at time zero.

A solution S comprises an ordered route $\Gamma_a = (i_{a,1}, \dots, i_{a,n_a})$ for each platform and an unassigned task set $U(S)$. Together, the routes and $U(S)$ partition $\mathcal{T}$, ensuring unique assignment. Each route includes departure from and return to the base, with task-to-platform assignments and visit orders determined jointly.

Feasibility combines capability matching, timing, and transfer energy. Every assigned task requires $\mathcal{R}_{i_{a,r}} \subseteq \mathcal{C}_a$. With Euclidean distance d_{ij} , travel time is $\tau_{ij}^a = d_{ij}/v_a$. Set $i_{a,0} = 0$ and $b_{a,0} = s_0 = 0$. The service-start time at each visit is

$$b_{a,r} = \max\{e_{i_{a,r}}, b_{a,r-1} + s_{i_{a,r-1}} + \tau_{i_{a,r-1}, i_{a,r}}^a\}, \quad b_{a,r} \leq l_{i_{a,r}}. \quad (1)$$

Early arrivals wait until the window opens; the upper bound constrains service start. The transfer energy for departure, intertask travel, and return is

$$E_a(\Gamma_a) = \varepsilon_a \left(\sum_{r=1}^{n_a} d_{i_{a,r-1}, i_{a,r}} + d_{i_{a,n_a}, 0} \right) \leq E_a^{\max}. \quad (2)$$

Empty routes consume zero energy, and waiting and service contribute only to the time calculation.

Solutions are ranked by the fixed three-level lexicographic objective

$$\min_S^{\text{lex}}(L_1, L_2, L_3), \quad L_1 = \sum_{i \in U(S)} p_i, \quad L_2 = |U(S)|, \quad L_3 = \sum_{a \in \mathcal{A}} E_a(\Gamma_a). \quad (3)$$

Minimizing priority loss L_1 favors assigning higher-priority tasks. At equal L_1 , minimizing the unassigned task count L_2 favors including more tasks; at equal L_1 and L_2 , minimizing L_3 reduces total transfer energy.

3. THE IntALNS METHOD

IntALNS couples interpretable LLM-agent decisions with adaptive large neighborhood search. Allocation observations, execution history, and strategy examples inform neighborhood controls and their stated rationales; ALNS reconstructs routes, enforces feasibility, and returns outcomes for subsequent decisions (Fig. 1). The current and historical-best solutions and adaptive search state persist across windows.

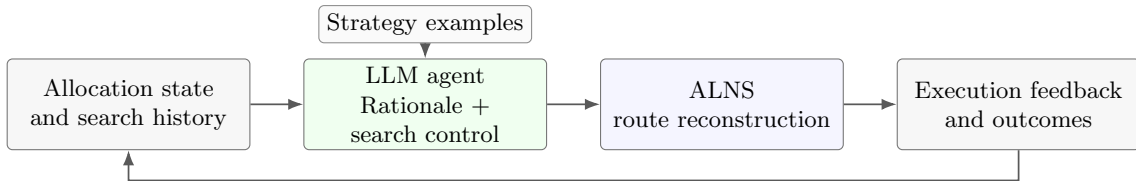


Figure 1. Allocation observations, history, and strategy examples inform LLM rationales and search controls; ALNS executes the controls and returns feedback.

3.1 Interpretable Search Decisions

The allocation observation o_t summarizes the search state through current and historical-best objective values, unassigned tasks, compatible platforms, feasible insertion positions, and time and energy margins. The agent uses this information to assess the current allocation and select the search focus and neighborhood controls for the next window.

At search window t , the agent combines o_t with execution history $\mathcal{H}_t$ and retrieved strategy examples $\mathcal{E}_t$ to choose $\mathbf{u}_t$, which specifies the search focus, operator preferences, removal strength, and acceptance setting. The decision is expressed as

$$\mathbf{u}_t = \text{LLM}(o_t, \mathcal{H}_t, \mathcal{E}_t), \quad (4)$$

Execution history $\mathcal{H}_t$ records the controls applied in earlier windows together with their observed objective changes, operator use, acceptance outcomes, and insertion results. It supplies the agent with evidence of how previous search adjustments performed and whether allocation progress has continued or stalled. Relating this feedback to the current observation supports decisions about retaining a productive direction, changing operator preferences, or increasing the extent of route reconstruction.

Strategy examples $\mathcal{E}_t$ are retrieved from development instances according to task scale and search situation. Each example pairs a previous decision with its execution outcome, providing a concrete reference for addressing a related allocation difficulty. The examples broaden the decision context with experience from other searches, while $\mathcal{H}_t$ supplies feedback from the ongoing run; together, they inform controls suited to the current state.

Interpretability comes from grounding the stated rationale in recorded evidence. For subsequent decisions, the agent must reference actual historical feedback, such as objective changes or insertion results, and explain how it motivates the proposed control. Retaining this rationale alongside the observation, executed control, and subsequent outcome lets readers trace which feedback informed the adjustment and how the stated decision was carried out. This makes the basis of search control explicit.

3.2 Agent-Controlled Destroy-and-Repair Search

Within each window, the agent’s operator preferences combine with ALNS adaptive weights to select destroy and repair operators. Preferences reflect the current allocation state, while adaptive weights reflect earlier operator performance. Removal strength sets the number of tasks removed; destroy operators select them randomly or by local route cost, spatial proximity, or related time windows.

Removed and previously unassigned tasks form the repair pool. The search focus guides stochastic task selection, either across the pool or toward tasks with limited feasible insertion opportunities. Repair then chooses the feasible position with the smallest incremental transfer energy or samples a feasible position randomly. Reassessing insertion opportunities as routes change allows reconstruction to revise task assignments and visit orders.

Candidates follow the lexicographic objective in Eq. (3). Improvements in (L_1, L_2) are accepted and deteriorations rejected. At equal L_1 and L_2 , greedy acceptance permits no increase in L_3 , while threshold or simulated-annealing acceptance can admit temporary increases under the selected setting.¹² The historical-best solution is retained separately from the current solution. Each window’s objective changes and search outcomes are added to $\mathcal{H}_t$ for the next decision. Algorithm 1 summarizes the cycle.

Algorithm 1 IntALNS

Input Instance I , feasible initial solution S_0 , search budget B , control interval h

1. $S \leftarrow S_0$; $S^* \leftarrow S_0$; $\mathcal{H}_0 \leftarrow \emptyset$; initialize adaptive search state
 2. **for** $t = 0, \dots, B/h - 1$ **do**
 3. Form o_t from I , S , S^* , insertion opportunities, and remaining budget
 4. Retrieve $\mathcal{E}_t$; obtain and validate $\mathbf{u}_t$ using Eq. (4)
 5. **for** $j = 1, \dots, h$ **do**
 6. Sample operators using $\mathbf{u}_t$ and adaptive weights; remove and repair tasks to form S'
 7. Evaluate S' ; update S by the acceptance rule, S^* by Eq. (3), and search statistics
 8. **end for**
 9. Append $\mathbf{u}_t$ and execution feedback to $\mathcal{H}_t$ to form $\mathcal{H}_{t+1}$
 10. **end for**
 11. **return** S^*
-

4. EXPERIMENTS

4.1 Experimental Environment

The LLM agent uses DeepSeek-V4-Flash, a mixture-of-experts language model supporting both thinking and non-thinking modes. Experiments disable additional reasoning and set the sampling temperature to 0.

4.2 Dataset Generation

Synthetic instances contain six heterogeneous platforms and 50, 100, or 300 tasks, denoted T50, T100, and T300. Task locations, requirements, service durations, time windows, and platform resources are generated jointly from complete, capability-compatible reference routes. The service-start time windows accommodate the reference visits, and energy budgets exceed reference-route transfer energy. Each instance therefore admits a complete feasible allocation. Five instances per scale give 15 instances. Within each instance, one random permutation selects nested task subsets with initial allocation ratios of 10%, 50%, and 90%, denoted M10, M50, and M90. Initial routes preserve the relative order of retained tasks; the remaining tasks are unassigned. Search may reorder or reassign tasks, giving 45 instance-start conditions.

4.3 Task Allocation Performance

Basic is an adaptive ALNS baseline without LLM search control; it updates operator probabilities using historical rewards. Both methods share the initial solutions, operator library, and constraint handling. Basic’s parameters were selected on a small separate tuning set, with one configuration per task count. Both methods use algorithm seed 0 and 1200 iters; best solutions are recorded every 100 iters, when IntALNS also updates its controls.

Mean L_1 and L_2 cover all conditions, and wins/ties/losses (W/T/L) compare IntALNS against Basic under the full lexicographic objective. Mean L_3 uses the subset defined in Table 1. A complete allocation assigns every task, giving $L_1 = L_2 = 0$.

IntALNS produces solutions with lower priority loss and more complete allocations. Across the 45 conditions, mean priority loss decreases from 3.62 to 1.82, a reduction of 49.69%, while complete allocations increase from 16 for Basic to 25 for IntALNS. Mean unassigned task count also falls, from 2.24 to 1.40. Under the full lexicographic objective, IntALNS records 30 wins, 5 ties, and 10 losses (Table 1). The aggregate improvement thus includes both reduced priority loss and fewer omitted tasks.

Table 1. Final task allocation performance and conditional transfer energy.

Size	Mean L_1		Mean L_2		Mean L_3 ($\times 10^3$) [†]		W/T/L
	Basic	IntALNS	Basic	IntALNS	Basic	IntALNS	
T50	1.40	0.53	0.80	0.47	31.66	30.25	9/4/2
T100	2.60	1.20	1.87	0.87	58.09	57.80	11/1/3
T300	6.87	3.73	4.07	2.87	168.02	168.56	10/0/5
All	3.62	1.82	2.24	1.40	72.36	71.72	30/5/10

[†] Mean L_3 uses pairs with equal L_1 and L_2 ($n = 10, 5, 5$ for T50, T100, T300; 20 overall). Displayed energy values are divided by 10^3 . Bold marks lower means.

The scale-specific results highlight two aspects of this improvement. T300 has the largest absolute decrease in mean priority loss, from 6.87 to 3.73, as the same fleet handles the largest task set. At T100, complete allocations increase from 3 to 8 of the 15 conditions, showing that more searches resolve all remaining assignments. Together, these findings indicate better protection of important monitoring tasks and a greater ability to include the full task set in a feasible solution.

The advantage develops at different stages of the search. At T100 and T300, IntALNS reaches lower mean priority loss and unassigned task count by 100 iters and maintains both gaps throughout the search. At T50, IntALNS initially leaves more tasks unassigned, and its advantage emerges later, first in priority loss and then in unassigned task count. These trajectories connect the final allocation gains to improvements developed during search (Fig. 2).

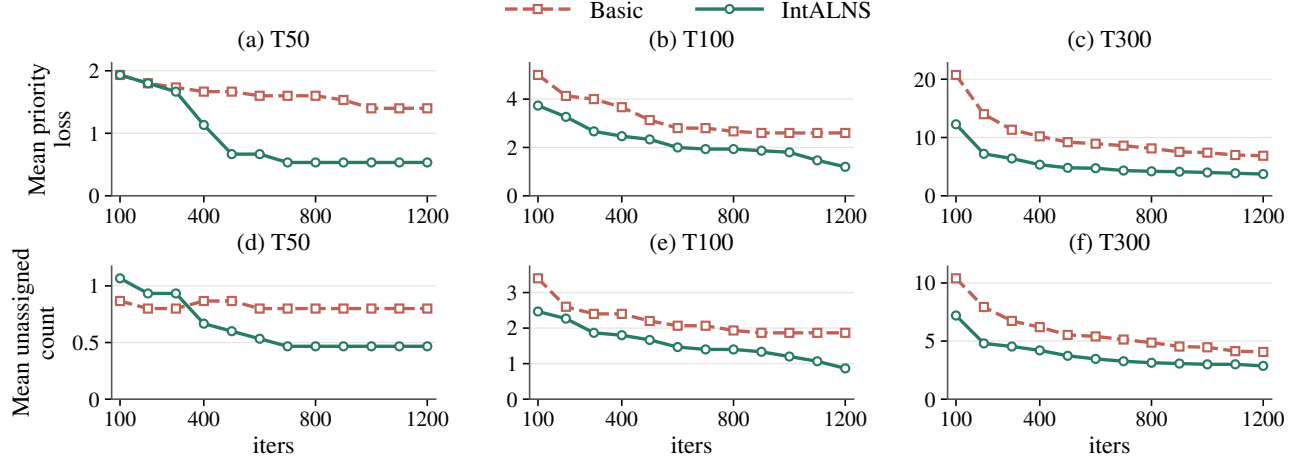


Figure 2. Mean priority loss and unassigned task count of the lexicographic-best solutions across the 15 instance-start conditions at each task scale.

Among pairs with matching L_1 and L_2 , IntALNS achieves lower mean L_3 on T50 and T100, whereas T300 shows a small increase (Table 1). This increase alone does not indicate poorer allocation performance overall, because equal L_1 and L_2 can correspond to different task assignments and routes with different transfer-energy costs.

4.4 Interpretability Analysis

To examine the interpretability of IntALNS decisions, we trace one search transition from the observed allocation state to the agent rationale, executed control, and subsequent outcome. At 600 iters in T100, instance 6800, M50, (L_1, L_2) had remained unchanged for 5 consecutive search windows, while L_3 decreased in the latest window. All 4 unassigned tasks lacked a feasible insertion position. The agent interpreted this combination as a need to release assigned tasks that blocked further allocation and proposed broader route reconstruction. It increased the removal ratio from 0.4 to 0.5, keeping the other controls unchanged. The excerpts in Table 2 link the observed bottleneck to this rationale and the executed control.

By 700 iters, (L_1, L_2) improved from (8, 4) to (7, 3) (Fig. 3). The recorded rationale identifies stalled task allocation as the reason for increasing removal strength, making the link between the observed search state and the executed control explicit.

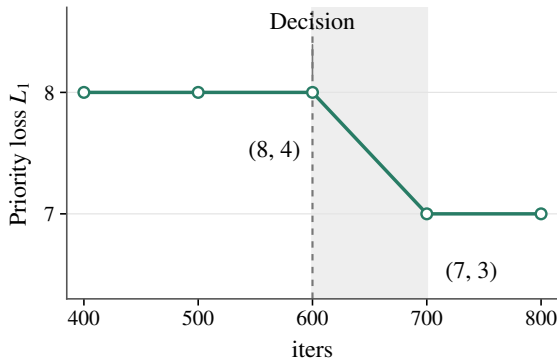


Figure 3. Neighborhood adjustment following task-allocation stagnation. Shading marks 600–700 iters after the control adjustment; labels give (L_1, L_2) .

Table 2. Observed bottleneck, decision rationale, and executed control.

<i>Observed allocation and feedback</i>	
"zero_feasible_task_count":	4
"consecutive_windows_without_service_progress":	5
"energy_improved_at_equal_service":	true
<i>Agent rationale</i>	
"Service has stalled for five windows at [8,4] with all four unassigned tasks showing zero feasible positions, so probe a larger .50 destruction (about 48 tasks) to release coupled assigned blockers"	
<i>Executed control</i>	
"remove_ratio":	0.5
"focus_id":	"service_bottleneck"

5. CONCLUSION

We propose IntALNS for cooperative marine monitoring, formulating task allocation and route sequencing under platform capability, service-start time-window, and transfer-energy constraints. We design an agent-control mechanism that integrates allocation observations, execution history, and strategy examples to guide feasible ALNS route reconstruction. Requiring rationales to cite actual historical feedback makes search adjustments interpretable, as illustrated by the recorded case. Across 45 synthetic conditions, IntALNS reduces mean priority loss by 49.69% and increases complete allocations from 16 to 25 compared with Basic under the same search budget. IntALNS thus combines better allocation of important monitoring tasks with search control grounded in recorded evidence.

REFERENCES

- [1] Bi, W., Zhang, M., Chen, H., and Zhang, A., “Cooperative task allocation method for air-sea heterogeneous unmanned system with an application to ocean environment information monitoring,” *Ocean Eng.* **309**, 118496 (2024).
- [2] Liu, Y. and Bucknall, R., “Efficient multi-task allocation and path planning for unmanned surface vehicle in support of ocean operations,” *Neurocomputing* **275**, 1550–1566 (2018).
- [3] Nunes, E., Manner, M., Mitiche, H., and Gini, M., “A taxonomy for task allocation problems with temporal and ordering constraints,” *Robot. Auton. Syst.* **90**, 55–70 (2017).
- [4] Choi, H.-L., Brunet, L., and How, J. P., “Consensus-based decentralized auctions for robust task allocation,” *IEEE Trans. Robot.* **25**(4), 912–926 (2009).
- [5] Wang, F., Zhao, L., and Paik, J. K., “Modeling and solving time-sensitive task allocation for USVs with mixed capabilities,” *Ocean Eng.* **313**, 119614 (2024).
- [6] Ropke, S. and Pisinger, D., “An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows,” *Transp. Sci.* **40**(4), 455–472 (2006).
- [7] Zhou, J., Wu, Y., Cao, Z., Song, W., Zhang, J., and Chen, Z., “Learning large neighborhood search for vehicle routing in airport ground handling,” *IEEE Trans. Knowl. Data Eng.* **35**(9), 9769–9782 (2023).
- [8] Reijnen, R., Zhang, Y., Lau, H. C., and Bukhsh, Z., “Online control of adaptive large neighborhood search using deep reinforcement learning,” in *[Proc. Int. Conf. Automated Planning and Scheduling]*, **34**(1), 475–483 (2024).
- [9] Yang, C., Wang, X., Lu, Y., Liu, H., Le, Q. V., Zhou, D., and Chen, X., “Large language models as optimizers,” in *[Int. Conf. Learning Representations]*, (2024).
- [10] Ye, H., Wang, J., Cao, Z., Berto, F., Hua, C., Kim, H., Park, J., and Song, G., “ReEvo: Large language models as hyper-heuristics with reflective evolution,” in *[Adv. Neural Inf. Process. Syst.]*, **37**, 43571–43608 (2024).
- [11] Zhang, J., Luo, C., Su, Z., Zhang, Q., Lü, Z., Ding, J., and Jin, Y., “NS4S: Neighborhood search for scheduling problems via large language models,” in *[Proc. 34th Int. Joint Conf. Artificial Intelligence]*, 8687–8695, IJCAI (2025).
- [12] Santini, A., Ropke, S., and Hvattum, L. M., “A comparison of acceptance criteria for the adaptive large neighbourhood search metaheuristic,” *J. Heuristics* **24**(5), 783–815 (2018).